## Language

### Автор сайта

Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс
Jetpack Compose

### Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

### Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

## Поддержка проекта

Яндекс
**410011180491924**

WebMoney
**R248743991365**
**Z551306702056**

Paypal

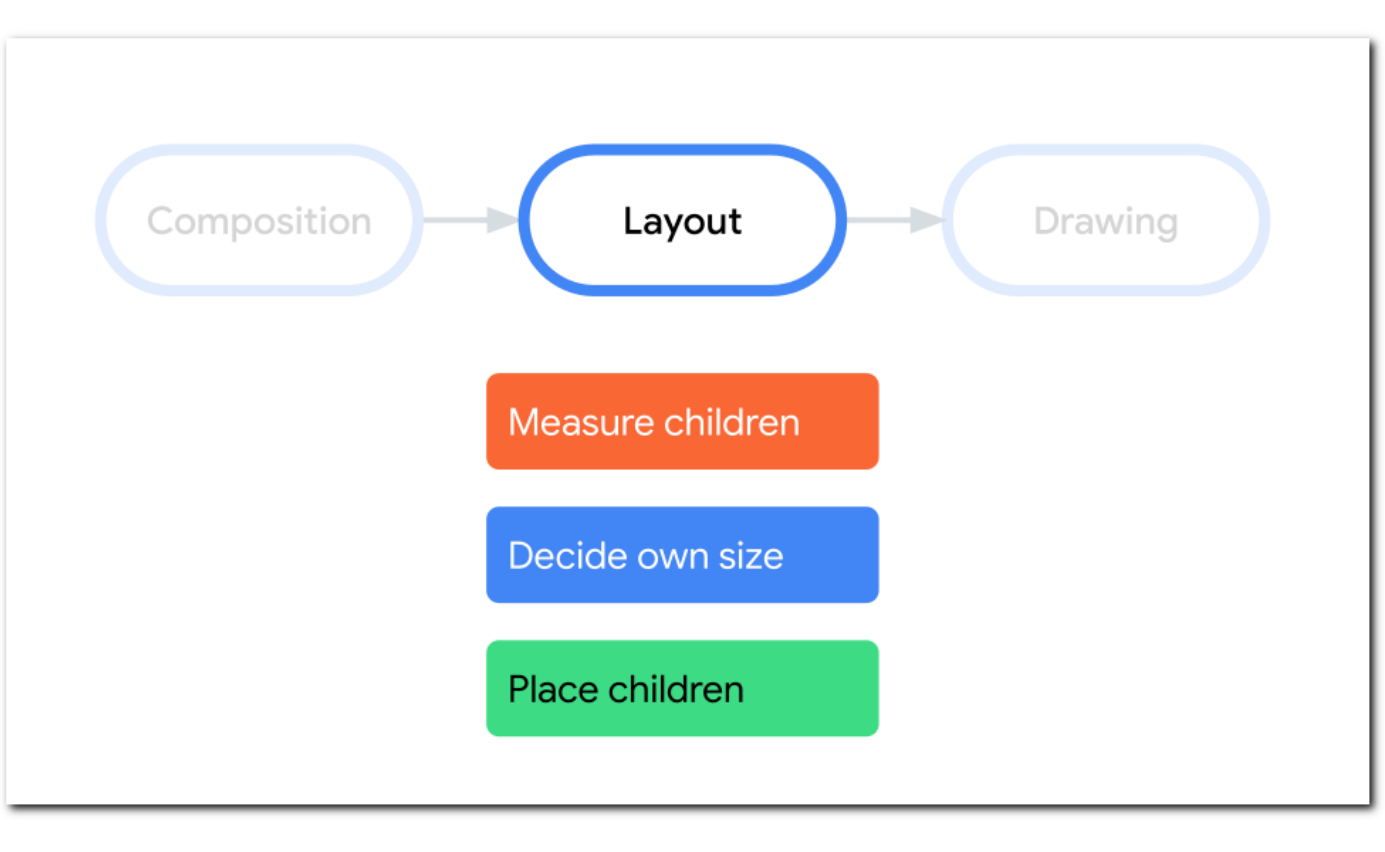
# Урок 25. Custom Layout

28 июня 2023

В этом уроке создаем свой кастомный Layout - простую версию Column

На примере LayoutModifier мы узнали, как элементы Layout иерархии обмениваются данными.

Вспомним эту схему



Measure children - родитель опрашивает дочерние UI элементы об их размере
Decide own size - в зависимости от результатов родитель вычисляет собственный размер
Place children - родитель размещает дочерние UI элементы

То же самое делают и Layout типа Column или Row. Отличие от LayoutModifier в том, что у них может быть не один, а несколько дочерних элементов.

Давайте создадим простой аналог Column, чтобы посмотреть, как это работает:

```
@Composable
fun MyColumn(
   modifier: Modifier = Modifier,
   content: @Composable () -> Unit
) {
   Layout(
       modifier = modifier,
       content = content
   ) { measurables, constraints ->
       val placeables = measurables.map {measurable ->
           measurable.measure(constraints) // Measure children
       }
       layout(constraints.maxWidth, constraints.maxHeight) { // Decide own size
           var y = 0
           placeables.forEach { placeable ->
               placeable.placeRelative(x = 0, y = y) // Place children
               y += placeable.height
           }
       }
   }
}
```

Вызываем функцию Layout, в которую передаем modifier и content. content - это слот, в который мы будем помещать дочерние элементы при использовании MyColumn в приложении.

Смотрим код внутри Layout. Он похож на то, что мы делали в ModifierLayout.

Т.к. дочерних элементов может быть несколько, то вместо одного measurable нам приходит коллекция - measurables. Каждый элемент в ней мы просим посчитать его размеры с учетом constraints. Это дает нам коллекцию placeables.

```
val placeables = measurables.map {measurable ->
    measurable.measure(constraints) // Measure children
}
```

В качестве своего размера мы пока передаем просто максимально доступные нам размеры из constraints.

```
layout(constraints.maxWidth, constraints.maxHeight) { // Decide own size
```

Чуть позже мы это поменяем

Интересное начинается в момент определения координат дочерних элементов:

```
var y = 0
placeables.forEach { placeable ->
    placeable.placeRelative(x = 0, y = y) // Place children
    y += placeable.height
}
```

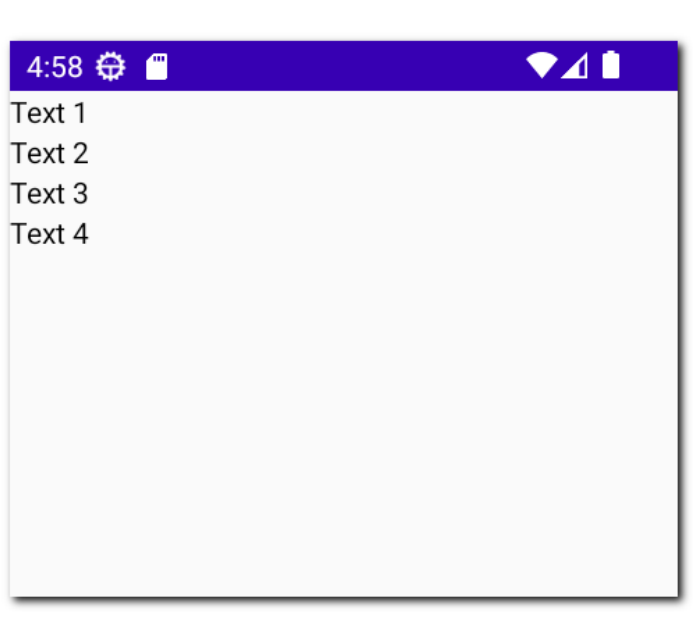
Напомню, что тут мы каждому элементу сообщаем его будущие координаты внутри нашего кастомного Layout. И т.к. мы делаем аналог Column, мы должны размещать элементы по вертикали, друг под другом. Это значит, что Y координата каждого следующего элемента будет равна Y предыдущего элемента + высота предыдущего элемента.

Именно это мы и делаем в коде. Каждый элемент увеличивает Y на значение своей высоты. Тем самым он смещает вертикально вниз позицию следующего элемента. В итоге элементы выстроятся в вертикальный ряд.

Давайте проверим, как работает наш кастомный Layout:

```
@Composable
fun HomeScreen() {
   MyColumn {
       Text("Text 1")
       Text("Text 2")
       Text("Text 3")
       Text("Text 4")
   }
}
```

Результат:



Text 1
Text 2
Text 3
Text 4

Тексты выстроились вертикально, как мы и планировали.

Вернемся к размерам нашего MyColumn

Мы в качестве своих размеров передавали родителю максимально доступные нам значения

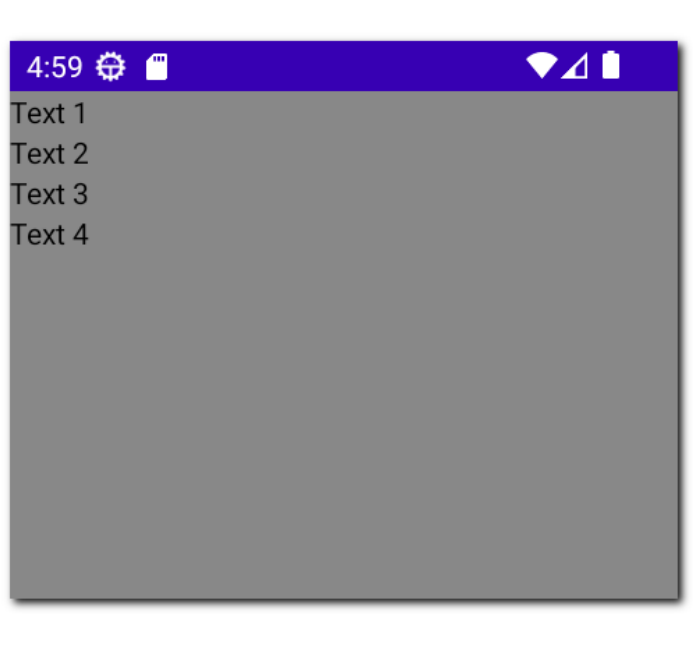
```
layout(constraints.maxWidth, constraints.maxHeight) { // Decide own size
```

В итоге MyColumn займет все доступное ему пространство, не зависимо от размера своих дочерних элементов.

Добавим серый фон, чтобы убедиться:

```
@Composable
fun HomeScreen() {
   MyColumn(modifier = Modifier.background(Color.Gray)) {
       Text("Text 1")
       Text("Text 2")
       Text("Text 3")
       Text("Text 4")
   }
}
```

Результат:



Text 1
Text 2
Text 3
Text 4

Давайте исправим это. Сделаем так, чтобы MyColumn занимал ровно столько места, сколько нужно его дочерним элементам, как это делает стандартный Column. Для этого нам надо посчитать ширину и высоту, используя данные о размерах дочерних элементов - placeables

Ширина должна быть равна ширине самого широкого дочернего элемента:

```
placeables.maxOf { it.width }
```

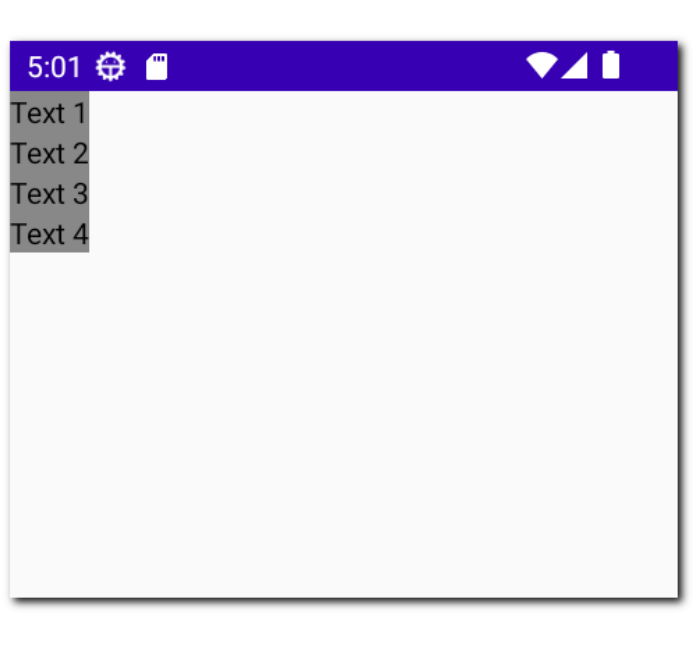
А высота будет равна суммарной высоте всех дочерних элементов:

```
placeables.sumOf { it.height }
```

Передаем эти данные в layout:

```
Layout(
   modifier = modifier,
   content = content
) { measurables, constraints ->
   ...
   layout(placeables.maxOf { it.width }, placeables.sumOf { it.height }) { // Decide own size
       ...
   }
}
```

Результат:



Text 1
Text 2
Text 3
Text 4

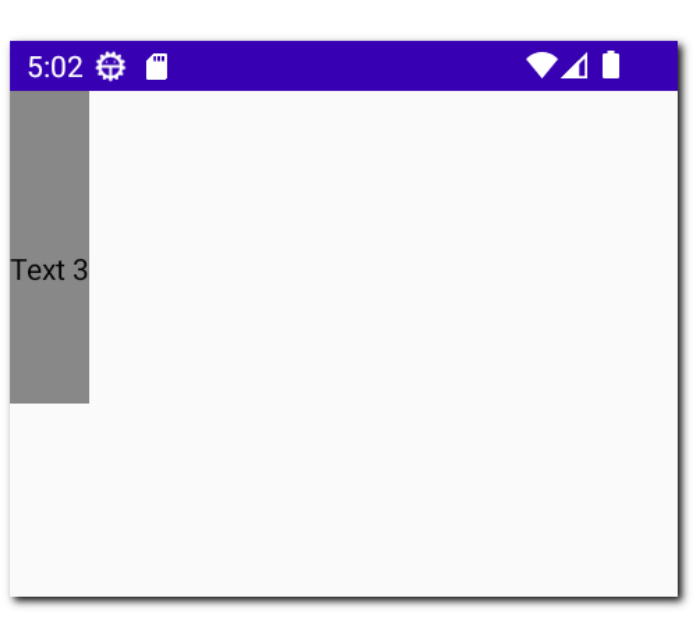
Теперь MyColumn занимает ровно столько пространства, сколько нужно, чтобы вместить все дочерние элементы.

MyColumn получился очень простой и его очень легко сломать.

Например, давайте добавим к нему фиксированную высоту:

```
@Composable
fun HomeScreen() {
   MyColumn(modifier = Modifier.height(150.dp).background(Color.Gray)) {
       Text("Text 1")
       Text("Text 2")
       Text("Text 3")
       Text("Text 4")
   }
}
```

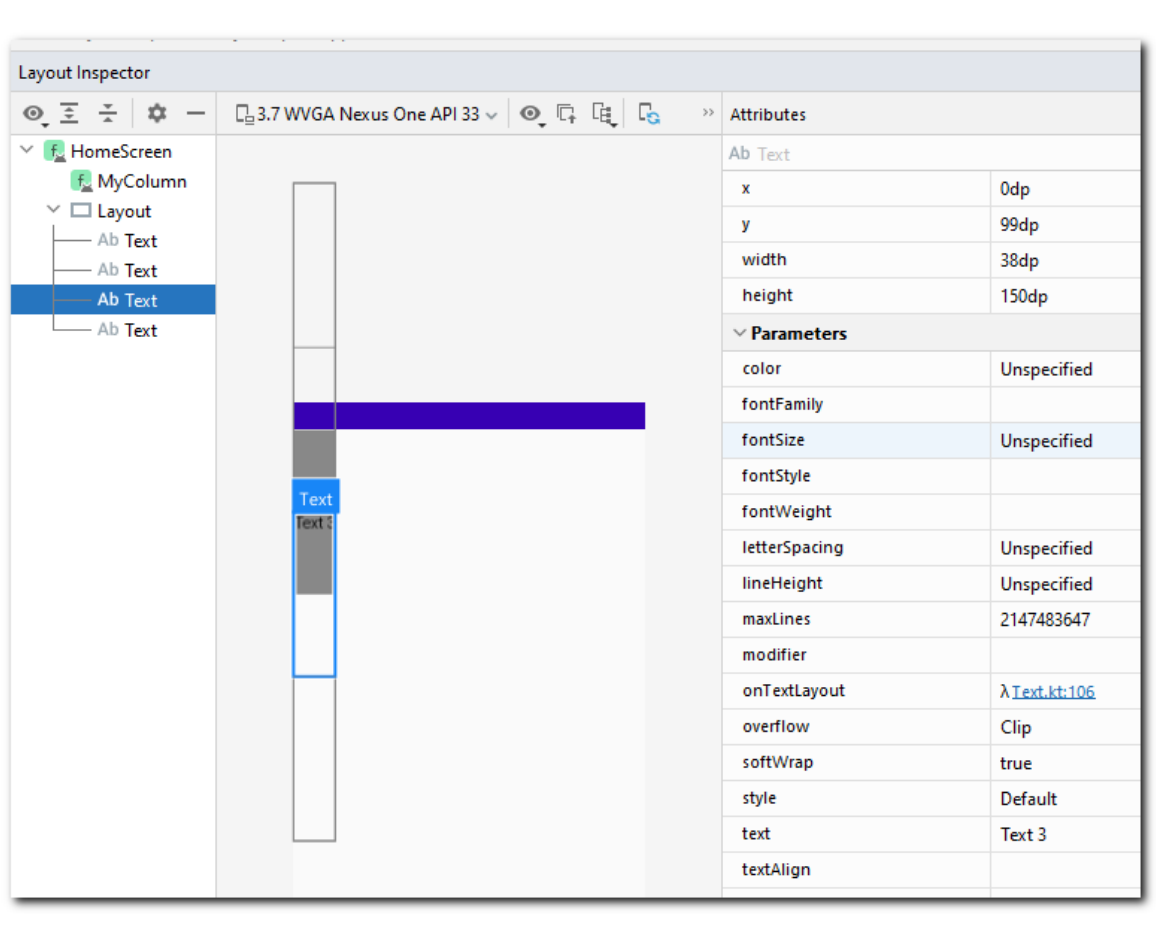
Результат:



Text 3

Не очень понятно, что происходит.

Посмотрим на это в Layout Inspector:



| | |
|---|---|
| x | 0dp |
| y | 99dp |
| width | 38dp |
| height | 150dp |
| color | Unspecified |
| fontFamily | |
| fontSize | Unspecified |
| fontStyle | |
| fontWeight | |
| letterSpacing | Unspecified |
| lineHeight | Unspecified |
| maxLines | 2147483647 |
| modifier | |
| onTextLayout | λ Text.kt:106 |
| overflow | Clip |
| softWrap | true |
| style | Default |
| text | Text 3 |
| textAlign | |

Видно, что высота 150 dp применилась к каждому тексту. Все дело в constraints.

Добавление height(150.dp) привело к тому, что в MyColumn приходит constrains, который говорит, что минимальная и максимальная высота должна быть 150.dp. А мы этот же constraints передаем всем дочерним текстам (measurables). В итоге все тексты думают, что они должны быть высотой в 150.dp

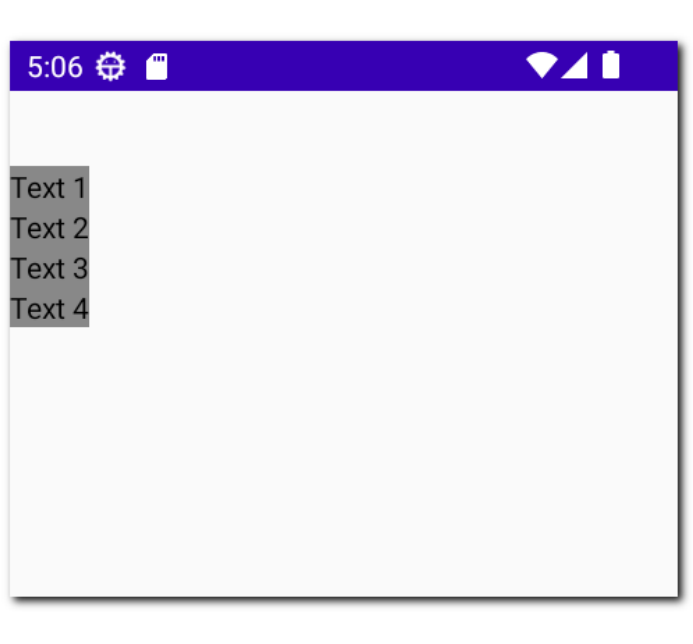
Это можно поправить. Мы можем сказать текстам, что 150.dp - это максимально доступная им высота. А минимальная высота - 0. В этом случае тексты не будут принудительно себя растягивать до 150.dp.

Реализуем:

```
Layout(
   modifier = modifier,
   content = content
) { measurables, constraints ->
   val childConstraints = constraints.copy(minWidth = 0, minHeight = 0)
   val placeables = measurables.map {measurable ->
       measurable.measure(childConstraints) // Measure children
   }
   ...
}
```

Мы создаем копию полученного constraints, в которой сбрасываем на 0 минимальные высоту и ширину. И передаем эти ограничения дочерним текстам.

Результат:



Text 1
Text 2
Text 3
Text 4

Частично помогло, теперь тексты имеют адекватную высоту. Но почему-то содержимое MyLayout сместилось вниз.

Причина снова в constraints, только теперь не в сторону дочерних элементов, а в сторону родителя нашего MyLayout. Родитель сказал, что высота должна быть 150.dp, а мы ему вернули высоту, которая равна суммарной высоте всех текстов. Это около 80.dp

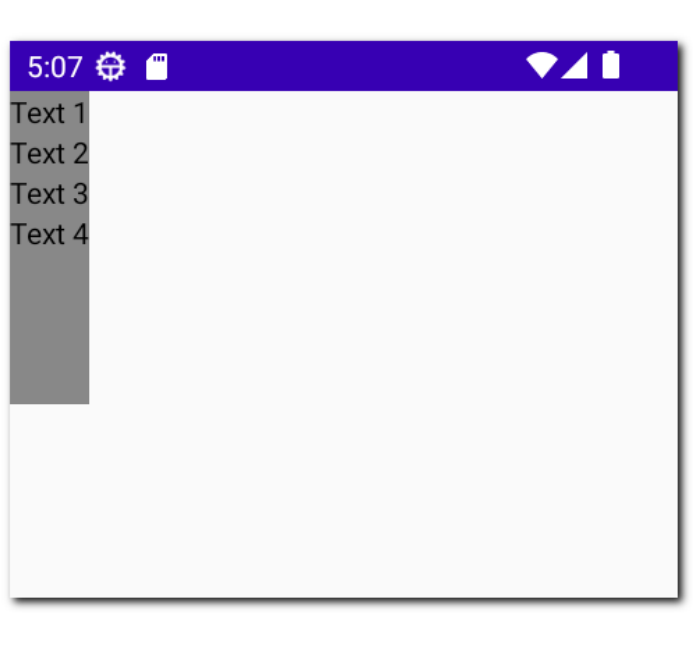
Это и привело к такому искажению. 80.dp реальной высоты просто расположились по центру по вертикали в 150.dp требуемой высоты.

Чтобы возвращать родителю адекватные размеры, используем специальные функции в constraints:

```
layout(
   constraints.constrainWidth(placeables.maxOf { it.width }),
   constraints.constrainHeight(placeables.sumOf { it.height })
) { // Decide own size
```

Размеры которые мы рассчитываем по дочерним элементам мы оборачиваем в constrain* функции. Они подгонят наши размеры в соответствие с ограничениями constraints и на выходе мы получим результат, максимально близкий к тому, который ожидает от нас родитель нашего MyColumn.

Результат:



Text 1
Text 2
Text 3
Text 4

Стало гораздо лучше.

Как видите, создавать свой Layout - дело непростое, надо быть аккуратным и проверять, как оно будет работать в комбинации с различными Modifier операторами.

Также учитывайте, что дочерние элементы нельзя спрашивать более чем один раз об их размерах. Это приведет к крэшу:
*java.lang.IllegalStateException: measure() may not be called multiple times on the same Measurable*

Но если вдруг такая необходимость все таки возникла, используйте Intrinsic.

---

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

---

Назад Вперёд

### Комментарии

# **Передача параметров в content** — **Pavlo** 22.01.2024 23:44
Дмитрий, возник вопрос.
Как можно передать данные в контент функции

Layout(
modifier = modifier,
content = {
Text(text = text1)
Text(text = text2)
}
)

если эти данные зависят от результатов измерения?

Например есть некая строка TEXT и ее надо вывести в ввиде двух колонок например, text1 и text2 зависят от результатов измерения, text1 - тот текст, который поместится в видимую облать, все остальное - это text2
Ответить

# **RE: Передача параметров в content** — **Dmitry Vinogradov** 12.02.2024 20:30
Можно попробовать использовать TextMeasurer из Урока 27. Он подскажет какого размера получится текст.
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

### Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.